Project MirrorFit

Full-Body Virtual Try-On and Brand-Aware Fit Intelligence Platform

Advanced Technical and Business Documentation

Architecture | Fit Engine | Asset Pipeline | API and Data Design | Market and Unit Economics | Roadmap

Author: Sanjay | Version 2.0 | August 2026

Table of Contents

1. Executive Summary

MirrorFit is a full-body virtual try-on and fit-intelligence platform for apparel retail. It solves two coupled problems: customers cannot visualise how a garment looks on their own body before buying, and they cannot tell which size of which brand will actually fit them — because sizing is inconsistent across brands (a Medium in one label is a Large in another). MirrorFit addresses both with a single on-device computer-vision pipeline: real-time body tracking renders the garment on the customer's live image, while the same tracked landmarks feed a fit engine that estimates body measurements and maps them against each brand's actual size chart to recommend the correct size per brand.

The platform is delivered progressively: first as a browser application on any phone or PC camera, then as an in-store IoT smart mirror where a garment's unique ID (QR or RFID tag) automatically loads that exact product onto the customer's reflection. Live 360-degree rotation is a core platform requirement: as the customer physically turns in place, the garment remains correctly rendered through profile and back views in real time, achieved through 3D garment geometry, continuous body-yaw estimation, and depth-assisted tracking on the mirror appliance. All video processing runs on-device; the cloud holds only the catalogue, brand size charts, and garment assets. This document specifies the technical architecture in depth, the business model, the build plan, current progress, and the forward roadmap.

2. Problem Statement

2.1 The visualisation gap

Apparel is bought on appearance, yet neither channel lets the customer see the product on their own body before committing. Online shoppers judge from photos of models with different bodies; in-store shoppers face fitting-room queues and try-on fatigue, evaluating only a handful of garments per visit. Both channels convert below their potential and apparel remains among the highest-return categories in Indian e-commerce, with size and appearance mismatch as the dominant return reasons.

2.2 The sizing chaos problem

There is no such thing as a universal Medium. Brand size charts differ by several centimetres for the same label size, vary between fits of the same brand (slim, regular, relaxed), and differ again between countries of origin. Customers compensate by ordering multiple sizes and returning the rest — a behaviour that is ruinous for retailer logistics and margins — or by sticking to the few brands whose fit they already know, which suppresses discovery of new brands. This is the specific problem MirrorFit's fit engine attacks: per-brand, per-garment size recommendation grounded in the customer's actual measured proportions.

2.3 The retailer blind spot

Physical retailers have no equivalent of e-commerce analytics. They cannot see which garments were considered and rejected, which were tried and abandoned, or which sizes were requested but unavailable. Every lost sale is invisible. A try-on platform instrumented at the point of decision produces exactly this data as a by-product.

2.4 Why current solutions fall short

- Entertainment-grade AR (social media filters) is not connected to real catalogues, sizes, or inventory.
- Imported smart-mirror hardware is closed and priced for luxury flagships, not Indian mid-market retail.

- Most try-on systems ignore fit entirely — they answer 'how does it look' but not 'which size do I buy', leaving the costliest half of the problem unsolved.
- Per-SKU manual 3D garment modelling collapses under real catalogue sizes and weekly drops, which is why many pilots never scaled.

3. Solution Statement and Differentiation

3.1 Solution statement

MirrorFit renders any catalogued garment on the customer's live camera image in real time, and simultaneously tells the customer which size of that specific brand fits them. The perception layer (body pose, segmentation, measurement estimation) runs entirely on-device in the browser. The knowledge layer (brand size charts, garment metadata, fit rules) lives in a lightweight cloud backend keyed by each garment's unique ID. The same application runs on a shopper's phone, a laptop, and — packaged in kiosk mode with a camera and portrait display — as an in-store smart mirror where tagged garments load automatically.

3.2 Differentiation

- **Fit intelligence, not just visualisation.** Look and fit in one pipeline: the landmarks that drive the garment render are the same signal that drives size recommendation. Competitors do one or the other; the combination is the product.
- **Brand-aware by design.** Size charts are ingested per brand and per fit-type, so recommendations are grounded in what the brand actually cuts, not a generic S/M/L heuristic. This is defensible: the size-chart corpus and its calibration against real return data compounds over time into a data moat.
- **Software-first delivery.** The same codebase serves shopper phones and store mirrors. Retail pilots start on a TV plus webcam with zero custom hardware, inverting the hardware-first cost structure of incumbent mirror vendors.
- **ID-triggered interaction.** A garment's physical tag (QR, later RFID) is the interface: pick up the dress, see it on yourself, see your size in it. No menus, no staff.
- **Privacy by architecture.** Video never leaves the device; only anonymised interaction events reach the cloud. Designed for DPDP compliance from day one, which is a procurement advantage with organised retail in India.
- **Retail intelligence layer.** Considered-but-not-bought analytics for physical stores, a data product that exists independently of the AR experience and monetises separately.

4. Technical Architecture

4.1 Per-frame perception and rendering pipeline

Each camera frame passes through the following stages on-device, targeting 30 fps end-to-end. The stages are decoupled so that perception (heavy, ML) can run at a lower frequency than rendering (light, GPU) with interpolation in between.

No.	Stage	Implementation	Output
1	Capture	getUserMedia, 1280x720 feed; 256x256 downscaled copy for ML	Video frame + tensor input

No.	Stage	Implementation	Output
2	Person detection	MediaPipe Pose detector stage	Person ROI
3	Pose estimation	MediaPipe Pose (BlazePose), 33 landmarks, world + normalised coords	Skeleton per frame
4	Segmentation	MediaPipe Selfie/Person segmentation	Person alpha mask
5	Temporal smoothing	One-Euro filter per landmark	Jitter-free skeleton
6	Measurement update	Rolling estimator over stable frames (see 4.4)	Body measurement vector
7	Garment fit + warp	Strategy-dependent (see 4.3)	Deformed garment layer
8	Compositing	WebGL: video, garment, mask-based occlusion	Final frame

Key engineering decisions: the One-Euro filter (rather than a plain lerp) gives low latency during fast motion and strong smoothing when still, which is exactly the trade-off body tracking needs. Segmentation enables occlusion — the customer's arms render in front of the garment when crossed — which is the single largest realism factor in user testing of comparable systems.

4.2 Body tracking subsystem

MediaPipe Pose provides 33 landmarks including shoulders (11, 12), elbows (13, 14), wrists (15, 16), hips (23, 24), knees (25, 26), and ankles (27, 28). Two coordinate sets are consumed: normalised image coordinates for rendering alignment, and metric world coordinates (hip-origin, metres) for measurement estimation. The world coordinates are scale-ambiguous from a monocular camera, so absolute scaling requires one calibration input (Section 4.4).

Model tiers are selected at runtime by benchmarking the device on load: lite (mobile, low-end), full (default), heavy (mirror appliance with GPU). Tracking confidence below threshold for 15 consecutive frames hides the garment layer and prompts the user to step back into frame — graceful degradation is treated as a product feature, not an error state.

4.3 Garment representation and rendering strategies

There is no single correct way to render cloth on a moving body in real time; the platform implements a tiered strategy where each garment declares its rendering tier in catalogue metadata, allowing cost and quality to be traded per SKU.

Tier 1: Landmark-warped 2D overlay (launch tier)

The garment is a high-resolution transparent PNG (front view, standardised pose) with an authored control mesh: a coarse triangular grid whose vertices are bound to body landmarks (shoulder points to landmarks 11/12, side seams interpolated between shoulder and hip, hem below hip midpoint). Each frame, the grid vertices follow the tracked landmarks and the texture warps by affine transform per triangle. Cheap, brand-scalable (any product photo can be prepared in minutes), and convincing for frontal use. Given the live-rotation requirement below, Tier 1 is explicitly an interim and long-tail tier: it launches the platform, remains valid for catalogue breadth, and hands off to Tier 2 wherever rotation matters. Its limitations — front view only (roughly plus/minus 35 degrees of body yaw), no drape physics — are surfaced honestly in the UI by fading the garment as the customer rotates beyond the valid range.

Tier 2: Cloth-simulated 3D garment (target tier)

The platform's target experience: true 3D garment models (CLO3D/Marvelous Designer export, or scan-derived), rendered in Three.js with a lightweight position-based-dynamics cloth solver on a low-poly proxy driven by the skeleton. Because the garment is genuine 3D geometry, it renders correctly from any viewing angle — which is what enables the live 360-degree rotation requirement below. Asset cost is 10-50x Tier 1, so rollout starts with hero SKUs and expands as the asset pipeline automates 3D conversion.

Live 360-degree rotation (core requirement)

The platform requirement is that the garment stays correctly rendered on the customer's live image as they physically rotate in place — front, profile, and back views, continuously. Rendering is the easy half: a Tier 2 garment posed by the skeleton is view-angle-agnostic. The hard half is tracking through the turn, and the design addresses it in four layers:

- **Continuous yaw estimation.** The body's yaw is computed each frame from the shoulder-hip landmark geometry (the shoulder vector's foreshortening and depth ordering), giving a continuous rotation signal that drives the garment's orientation even as individual limb landmarks degrade.
- **Confidence-gated bridging.** BlazePose accuracy drops in profile and back-facing poses, with occasional left-right landmark swaps. Each landmark's confidence is monitored; below threshold, the garment pose is driven by the yaw signal plus interpolation from the last high-confidence skeleton, then re-locks when the turn completes. Users perceive a smooth follow, not a glitch.
- **Swap suppression.** Left-right swap events are detected by temporal consistency checks (landmarks cannot teleport across the body between frames) and suppressed before they reach the renderer — the single most visible failure mode in rotation, handled explicitly.
- **Depth assist on appliance.** On the mirror appliance, the depth camera already planned for measurements doubles as a rotation aid: depth silhouettes disambiguate front-facing from back-facing poses, which monocular RGB fundamentally struggles with. On phones, rotation support is best-effort within RGB limits and the UI communicates this honestly.

One physical constraint is acknowledged by design rather than fought: when the customer faces fully away, they cannot see the screen. The live rendering continues throughout the turn (correct for observers and for the final moments of the rotation), and the completed turn is additionally captured and offered as an instantly scrubbable replay so the customer can inspect their own back view — a companion feature to, not a replacement for, live rotation.

Tier 3: Generative try-on (evaluation track)

Diffusion-based virtual try-on generates the dressed image directly from a product photo plus the person image, eliminating asset authoring entirely. Current constraints: per-image latencies well above real-time on affordable hardware, occasional garment hallucination (changed prints, invented buttons) which is commercially unacceptable for product representation, and GPU inference cost. Strategy: track the technology, benchmark quarterly, adopt for a 'photo mode' (non-real-time single shot) before live mode.

4.4 Brand-aware fit engine

The fit engine answers: for this customer, in this specific garment, which size should they buy, and how will it fit. It is the platform's core differentiator and runs in three stages.

Stage 1: Body measurement estimation

From metric world landmarks accumulated over stable frames (customer standing, facing camera, arms slightly apart — the UI coaches this pose), the engine derives: shoulder width (landmark 11-12 distance),

torso length (shoulder midpoint to hip midpoint), hip width (23-24 distance plus soft-tissue offset model), inseam (hip to ankle), and total height. Monocular scale ambiguity is resolved by one of three calibration paths, in order of preference: user-entered height (one field, highest accuracy), a reference object of known size (A4 sheet or credit card held to camera), or population-prior scaling (least accurate, fallback only). Circumference measurements (chest, waist) cannot be observed directly from one camera; they are estimated from width measurements via regression models fitted on public anthropometric datasets, and are reported with explicit confidence bands. The engine outputs a measurement vector with per-measurement confidence, refined continuously while the customer is in frame.

Stage 2: Brand size chart resolution

Every garment's unique ID resolves to a product record referencing a brand_size_chart: the brand's published garment measurements per label size, per fit type (slim/regular/relaxed), per category (shirts, tees, dresses, jeans), including tolerance and fabric stretch class. Charts are ingested from brand-published data via a normalisation pipeline into a canonical schema (all measurements in cm, defined measurement landmarks per category). Where a brand publishes body-measurement charts rather than garment measurements, category-standard ease allowances convert between the two. This corpus — normalised, per-fit, per-category charts across many brands — is deliberately treated as a growing strategic asset.

Stage 3: Fit scoring and recommendation

For each available size of the garment, the engine computes per-zone deltas (shoulder, chest, waist, hip, length) between the customer's measurement vector and the size's garment measurements minus category ease. Deltas map to a fit score per zone (too tight / snug / intended / loose / oversized) weighted by the garment's declared fit intent — a relaxed-fit tee is supposed to be loose. The output is a recommended size, a per-zone fit visual (rendered as a heat strip on the try-on UI), and a plain-language note ('Size M recommended: intended fit at chest, slightly long in sleeve'). Every recommendation is logged (anonymously) against eventual purchase and return outcomes where the retailer shares them, creating the feedback loop that calibrates the ease and soft-tissue models over time. Accuracy claims are deliberately conservative in v1: the engine recommends and explains, it does not guarantee.

4.5 Garment asset pipeline

Asset cost killed earlier generations of try-on products, so ingestion is an explicit pipeline, not an afterthought. For Tier 1: brand uploads front product photo (worn on mannequin or model, standardised); an automated stage segments the garment (removing mannequin/background), normalises resolution and colour against a reference chart, and generates the initial control mesh from garment landmarks detected on the image; a human-in-the-loop review tool (internal web app) lets an operator adjust control points in under two minutes per SKU; output (garment PNG + mesh JSON + metadata) versions into S3. Target ingestion cost: single-digit rupees per SKU at scale, enabling full-catalogue coverage — the property that makes 'every brand and every garment' feasible rather than aspirational. Tier 2 assets follow a separate authored workflow with per-asset QA.

4.6 Backend, API, and data model

The backend is deliberately thin: FastAPI (containerised on Fargate behind an ALB; Lambda acceptable at pilot scale), PostgreSQL for relational catalogue data (products, brands, size charts, stores), S3 + CloudFront for assets, and a separate event ingestion path (API Gateway to Kinesis Firehose to S3, queryable with Athena) for analytics events. Infrastructure is defined in AWS CDK. Core endpoints:

```
GET  /v1/garments/{uid}                -> product, brand, render tier, asset URLs
```

```
GET  /v1/garments/{uid}/sizes          -> sizes, stock (via retailer feed), chart
ref
POST /v1/fit/recommendation            -> {measurement_vector, garment_uid} -> size
+ zones
GET  /v1/brands/{id}/size-charts       -> normalised charts per category and fit
POST /v1/events                         -> anonymised interaction events (batched)
POST /v1/sessions/handoff              -> save-look QR: mirror session to phone
deep link
```

Data model (principal tables): brands; size_charts (brand_id, category, fit_type, size_label, measurements JSONB, ease JSONB); garments (uid, brand_id, category, render_tier, chart_id, asset_version); garment_assets (garment_uid, tier, s3_key, mesh_key, version); stores and store_inventory (optional retailer feed); events (session_hash, garment_uid, event_type, dwell_ms, ts — no biometric or identity fields by schema design). The measurement vector itself is never persisted server-side in the retail deployment; it lives in device session memory and is sent only transiently for the recommendation call, or the recommendation is computed fully client-side with charts fetched to the device — the latter is the default architecture.

4.7 Edge deployment (smart mirror appliance)

The mirror is the web application in Chrome kiosk mode on commodity hardware: a 43-55 inch portrait display (optionally behind two-way glass), a 1080p camera at 1.4-1.6 m height with the customer standing 2-2.5 m away, and an edge computer — a mini PC with an entry GPU or an Orin-class board, benchmarked to hold the heavy pose model at 30 fps. Garment ID capture starts as camera-based QR scan; the RFID upgrade adds a UHF reader so tagged garments are detected hands-free within range. Fleet management reuses standard DevOps practice: devices enrol via provisioning script, run a read-only OS image with the app as a container, report health metrics (fps, temperature, uptime, tracking quality) to Prometheus with Grafana dashboards and alerting, and receive updates via staged rollout. Offline resilience: the appliance caches the store's catalogue and charts locally and functions fully without connectivity except for live inventory.

4.8 Performance and latency budget

The perceived-quality threshold is motion-to-photon latency below roughly 100 ms and a stable 30 fps render. The per-frame budget on the mirror appliance:

Stage	Budget (ms)	Notes
Capture + downscale	3	GPU-accelerated
Pose inference (heavy model)	14	Every frame on appliance GPU
Segmentation	6	Can run alternate frames
Filtering + measurement update	1	CPU, trivial
Garment warp (Tier 1)	2	Per-triangle affine, GPU
Cloth solve (Tier 2 only)	8	PBD proxy, capped iterations
Composite + present	4	Single WebGL pass
Total (Tier 1 / Tier 2)	30 / 38	33 ms frame @ 30 fps

On mid-range phones, the lite pose model and alternate-frame segmentation keep Tier 1 within budget; Tier 2 is appliance-only. Fit recommendation is off the frame path entirely (computed once per garment, ~5 ms client-side).

4.9 Security, privacy, and DPDP compliance

Design commitments: no video frame or image leaves the device; no biometric template is created or stored (landmarks are ephemeral session state, cleared on session end); measurement vectors are session-scoped and, in the default architecture, never transmitted; analytics events carry a rotating session hash with no identity linkage; in-store deployments display clear signage and an on-screen indicator when tracking is active, with a visible opt-out gesture; the save-look handoff transfers only the composited image the customer explicitly requests, via QR to their own phone. Transport is TLS 1.3 throughout; assets are public-CDN but write-protected; the events pipeline is append-only with retention limits. A DPIA (data protection impact assessment) template is part of the retailer onboarding kit — turning compliance into a sales artefact.

5. Business Analysis

5.1 Market opportunity

India's apparel market is one of the largest globally and organised retail's share is growing, with thousands of mid-market fashion outlets in tier-1 and tier-2 cities — the deliberately chosen segment: large enough chains to buy technology, underserved by luxury-priced imported mirrors. Two adjacent markets multiply the opportunity: apparel e-commerce (phone-based try-on and fit SDK embedded in retailer apps) and the return-reduction economics that give the fit engine a measurable ROI story — every percentage point of size-related returns avoided is directly quantifiable money for an online retailer.

5.2 Competitive landscape

Player type	Strengths	MirrorFit's counter-position
Imported smart-mirror vendors	Mature hardware, luxury references	10x lower entry cost, software-first pilots, India-local support
Social AR platforms	Best-in-class tracking, huge reach	Not catalogue/commerce connected; no fit; not deployable in-store
Generative try-on startups	Zero asset cost, photorealism	Not real-time, product-fidelity risk; MirrorFit adopts as photo mode when mature
Size-recommendation SaaS	Fit focus, e-com integrations	Quiz-based, not measured; no visualisation; MirrorFit measures and shows
In-house retailer tech	Owns the channel	Build-vs-buy economics favour a platform across brands and stores

5.3 Revenue model and pricing

- **In-store SaaS.** Per-mirror monthly subscription covering software, fleet management, and support, with hardware either retailer-procured to spec or leased.

- **Catalogue onboarding.** Per-SKU ingestion fee at asset onboarding (covers pipeline cost with margin), waived above volume tiers to encourage full-catalogue coverage.
- **E-commerce SDK.** JavaScript SDK for retailer apps and websites: phone try-on plus size recommendation, priced per monthly active user or per recommendation call.
- **Retail intelligence.** Anonymised, aggregated engagement analytics dashboards for retailers; premium tier correlates try-on data with sales and return outcomes.

5.4 Unit economics (mirror deployment, indicative)

Item	One-time (INR)	Monthly (INR)
Display + mounting (retailer spec)	45,000-70,000	-
Edge compute + camera + QR	35,000-60,000	-
Installation + calibration	8,000-12,000	-
Connectivity + power	-	1,500
Fleet ops share (monitoring, updates)	-	2,000-3,000
Subscription price point (target)	-	12,000-20,000
Indicative gross margin on SaaS	-	60-70%

The retailer business case is framed on three levers: incremental conversion from engagement, basket growth from outfit suggestions, and the marketing value of the experience itself. The pilot's explicit job is to measure lever one credibly; pricing above assumes it lands.

5.5 Go-to-market strategy

- Phase A: one to two design-partner stores (single-brand outlets preferred — one size-chart ingestion, full catalogue coverage) on free pilots instrumented for engagement data.
- Phase B: convert pilots to paid, publish the case study, target regional mid-market chains via direct sales; simultaneously release the phone SDK to the pilot brands' e-commerce teams where return-reduction ROI is measurable fastest.
- Phase C: brand-side land-and-expand — a brand that ingests its size charts once can deploy across all its outlets and its app, making the size-chart corpus the cross-sell vehicle.

5.6 KPIs and success metrics

Dimension	Metric	Pilot target
Engagement	Try-on sessions per mirror per day	> 40
Engagement	Garments viewed per session	> 5
Experience	Session to physical-try conversion	> 30%
Fit engine	Recommendation acceptance rate	> 60%
Fit engine	Size-related return delta (e-com SDK)	measurable reduction
Technical	Median fps on appliance	30
Technical	Tracking-loss rate per session	< 2%
Business	Pilot to paid conversion	> 50%

6. Building Plan and Milestones

Phase	Scope	Key milestones	Duration
P1	Web try-on core	Pose + segmentation at 30 fps; One-Euro filtering; Tier 1 warp with 5 test garments; phone testing over HTTPS tunnel	6 weeks
P2	Fit engine v1	Height-calibrated measurement vector; chart schema + 3 brands ingested; recommendation API + fit-zone UI; accuracy study vs tape measurements (n=30)	5 weeks
P3	Catalogue + backend	FastAPI + Postgres + CDK deploy; asset pipeline with review tool; 200-SKU ingestion drill; events pipeline	4 weeks
P4	Kiosk pilot	Appliance image (kiosk Chrome, monitoring agent); QR garment trigger; save-look handoff; one design-partner store live	5 weeks
P5	Live 360 rotation	Tier 2 garment pipeline (3 hero SKUs); yaw estimation + swap suppression; confidence-gated bridging; depth camera integration on appliance; turn-replay capture	6 weeks
P6	Mirror v1 + scale	Two-way glass enclosure; RFID reader; analytics dashboards; fleet rollout tooling; second store	6-8 weeks

Phase gates

- P1 exit: Tier 1 overlay judged convincing by 10 non-technical testers on both laptop and mid-range Android.
- P2 exit: shoulder/height estimates within defined error bands vs tape measurement in the n=30 study; recommendation explains itself intelligibly.
- P4 exit: pilot store hits engagement KPI floor (Section 5.6) for four consecutive weeks.
- P5 exit: a full in-place turn renders without visible garment detachment or left-right flips in at least 9 of 10 test turns on the appliance, across 5 body types and 3 hero garments.

7. Current Status

- **Architecture and design complete.** This document: perception pipeline, tiered garment rendering strategy, fit-engine design, asset pipeline, API and data model, edge appliance design, latency budgets, and privacy architecture are specified.
- **Pipeline pattern validated in code.** The capture, on-device MediaPipe tracking, Three.js compositing, landmark anchoring, scaling, and smoothing pattern has been implemented and runs at real-time rates on a laptop webcam. Originally demonstrated on face tracking, the identical structure carries to

Pose — the porting work is swapping the model and rebinding anchors from face landmarks to body landmarks.

- **Market validation done.** Market landscape reviewed: commercial smart-mirror category active internationally and present in India, mid-market gap confirmed, RFID-triggered fitting-room pattern validated by existing deployments.
- **Next up.** Body-pose port, Tier 1 warp implementation, fit engine, size-chart ingestion, backend, and all in-store work are designed but not yet built. In roadmap terms the project stands at P1 start with the core rendering pattern de-risked.

8. Risks, Drawbacks, and Mitigations

Risk / drawback	Mitigation
Cloth non-rigidity: overlays show look, not physical drape	Tier strategy (2D for scale, 3D for heroes); position as discovery accelerator plus fit advisor, never a fitting replacement
Monocular measurement error, esp. circumferences	Height calibration required for recommendations; confidence bands surfaced; conservative claims; outcome-data calibration loop; depth camera option on appliance later
Tracking degradation during 360 rotation (profile/back poses, left-right swaps)	Yaw-driven garment orientation; confidence-gated bridging; temporal swap suppression; depth-assist on appliance; quality gate of 9/10 clean turns before launch; phone rotation labelled best-effort
Size-chart data quality varies by brand	Normalisation pipeline with validation rules; start with brands publishing garment measurements; flag low-confidence charts in UI
Asset pipeline throughput at real catalogue scale	Automation-first ingestion with human QA under 2 min/SKU; per-SKU cost tracked as a first-class KPI
Privacy perception of in-store cameras	On-device processing, zero retention, signage, active-indicator, DPIA onboarding kit; make privacy the sales pitch
Generative try-on leapfrogs the overlay approach	Quarterly benchmark; adopt as photo mode when fidelity/latency permit; fit engine and chart corpus remain differentiated regardless of render tech
Retailer ROI scepticism	Free instrumented pilots with pre-agreed KPI definitions; pricing gated on measured lift
Hardware fleet maintenance burden	Read-only OS images, containerised app, remote observability (Prometheus/Grafana), staged rollouts — standard SRE practice applied to retail devices

9. Future Roadmap and Design Direction

9.1 Near term (0-6 months)

- Execute P1-P3: web try-on with Tier 1 garments, fit engine v1 with three ingested brands, deployed backend, public demo URL.
- Measurement accuracy study and publication of honest error bands — credibility asset for retailer conversations.

9.2 Medium term (6-12 months)

- Kiosk pilot live in a design-partner store with QR trigger and analytics; e-commerce SDK alpha with one pilot brand's app.
- Live 360-degree rotation shipped on the appliance: Tier 2 hero garments, yaw-driven tracking through full turns, and the scrubbable turn-replay for back-view inspection.
- Outfit composition: multi-garment layering (top + bottom) with inter-garment occlusion ordering.
- Depth camera on the appliance serving double duty: direct metric measurement (removing height entry in-store) and back-facing pose disambiguation for rotation.

9.3 Long term (12+ months)

- Mirror v1 with RFID and two-way glass; multi-store fleet; retail intelligence dashboards as a paid tier.
- Photo-mode generative try-on once fidelity is commercially safe; evaluate for e-commerce PDP embedding.
- Fit data network effects: outcome-calibrated ease models per brand become the recommendation moat; explore cross-retailer size passport (user-owned measurement profile, consent-based).

9.4 Standing design principles

- Latency-critical work stays on-device; the cloud stays thin.
- Every phase ships something a retailer can see within weeks.
- Never misrepresent the product: colour fidelity and print accuracy are hard constraints on any rendering tier.
- Fit claims stay conservative until outcome data justifies stronger ones.

10. Demo Video Plan

Target: 2 minutes, recordable with the P1 build, a laptop, OBS Studio, and a phone. The fit-engine shots use the P2 build; until then a designed mock of the fit-zone UI stands in, clearly labelled as design.

Shot	Time	Content	Capture
1	0:00-0:12	Hook: sizing chaos — same person, three brands, three different 'correct' sizes; the return-parcel cost line	Stills + voiceover
2	0:12-0:35	Live try-on: person steps into frame, garment appears and follows movement; arms cross to show occlusion	OBS screen capture
3	0:35-0:55	Garment switching: three products swapped in seconds; phone shot showing same app on mobile	Screen + phone film
4	0:55-1:20	Fit engine: measurement capture pose, then per-zone fit strip and size recommendation with plain-language explanation	Screen capture / labelled mock
5	1:20-1:35	360 moment: person turns fully in place, garment follows through profile and back; turn-replay scrubbed on screen (P5 build; labelled concept animation until then)	Screen capture / animation
6	1:35-1:48	The mirror vision: tagged garment picked up,	Animated storyboard

Shot	Time	Content	Capture
		appears on reflection instantly; privacy line — nothing leaves the device	
7	1:48-2:05	Business close: discovery for shoppers, conversion and data for retailers; pilot call to action	Slides + voiceover

11. Summary

MirrorFit combines full-body virtual try-on with brand-aware fit intelligence in one on-device pipeline: the customer sees the garment on their own body and learns which size of that specific brand to buy. The technical core is deliberately pragmatic — proven on-device tracking, a tiered garment rendering strategy that scales to full catalogues, and a fit engine whose accuracy claims grow with outcome data rather than ahead of it. The business core is software-first delivery into India's mid-market apparel retail, monetised through mirror SaaS, an e-commerce SDK, and retail intelligence, with the normalised brand size-chart corpus compounding into the long-term moat. The rendering pattern is validated in working code; the immediate build priorities are the body-pose port, Tier 1 garment warping, and fit engine v1.